

Edge Detection Using Iterative Numerical Methods

Group 24

Amoako Ernestina Afriyie-1816322_20890206

Emmanuel Kwesi Annan - 1817122_20960733

Edmund Kwame Denteh - 1821522_20883173

Kofi Afriyie - 1814122_20894372

Abdallah Fatihu - 1812322_20916213

March 26, 2025

Contents

1	Introduction	4
2	Image Processing Overview	4
2.1	Definition And Importance	4
2.2	Types of Image Processing	4
2.3	Basic Steps In Image Processing	5
3	Edge detection In Image Processing	5
3.1	Edge Detection Techniques 	5
4	Laplace Equation and Edge Detection	6
4.1	Applications in Image Processing	6
4.2	Finite Difference Approximation	6
5	Iterative Methods for Solving the Laplace Equation	6
5.1	Jacobi Method	7
5.2	Gauss-Seidel Method	7
6	Implementation for Edge Detection	7
7	Challenges Faced	9
7.1	Iterative Smoothing and Edge Degradation	9
7.2	Computational Efficiency	9
7.3	Boundary Artifacts	10
8	Key Findings	10
8.1	Tolerance-Edge Sharpness Relationship	10
8.2	Method-Specific Convergence Patterns	11
8.3	Optimal Parameter Selection	11
9	CONCLUSION	12
10	References and Further Reading	15

Abstract

This report investigates the application of Jacobi and Gauss-Seidel iterative methods for solving the Laplace equation in edge detection. We demonstrate how these numerical techniques can process digital images to identify boundaries while analyzing their computational trade-offs. Key findings reveal unexpected relationships between tolerance values and edge sharpness, with higher tolerances (10^{-4}) producing superior results to strict tolerances (10^{-6}). The report includes quantitative convergence analysis and practical recommendations for implementation.

1 Introduction

Image processing begins with the phase of edge detection (ED), which comes before the step of identification of objects and is regarded to be the foundation of the processing. This approach includes the process of identifying these spots from images from the viewpoint of the pixels where a quick shift in brightness occurs. Images may be categorized into a wide range of distinct types, including as those utilised in medical, satellite images, articular imaging, industrial imaging, general-purpose imaging, as well as more

The most basic aspect of every image is its edges, which serve as boundaries between various parts and store a plethora of data. When it comes to image processing, ED is a common and crucial technique for detecting and distinguishing geometric forms . In most cases, ED is the method of locating the area of the graph in which the gradient undergoes an important modification and then picking the pixel that has the largest amplitude.

Additionally, it can get the edge data of an area by seeking out transfer locations in which the second-order derivative is equal to zero. ED operators typically consist of the Sobel operator, the Laplace operator, as well as the canny operator . Image ED has potential applications in data reduction, segmentation, picture reconstruction, and other wellmatched tasks. Making ED is possible in numerous ways.

This documentation focuses on using **iterative numerical methods** such as **Jacobi** and **Gauss-Seidel** to solve the **Laplace equation** for **edge detection** in image processing. The Laplace equation is a partial differential equation (PDE) that is widely used in computer vision and image processing to detect edges, which correspond to regions where pixel intensities change rapidly. By discretizing the Laplace equation and solving it iteratively, we can highlight edges in an image.

2 Image Processing Overview

2.1 Definition And Importance

Image processing is a field of computer science and engineering that focuses on the manipulation, analysis, and enhancement of images. It plays a crucial role in various applications, including medical imaging, remote sensing, computer vision, and digital photography. Image processing involves converting an image into digital form and performing operations to enhance or extract useful information. This process is essential in various fields such as:

- Medical Imaging: MRI and CT scan enhancements
- Remote Sensing: Satellite image analysis
- Computer Vision: Object recognition in autonomous vehicles
- Security Systems: Facial and fingerprint recognition

2.2 Types of Image Processing

1. Analog Image Processing: Used for manipulating hard copies of images using optical and electrical techniques.

2. Digital Image Processing: Uses computer algorithms to process digital images.

2.3 Basic Steps In Image Processing

- Image Acquisition: Capturing images using sensors.
- Image Enhancement: Adjusting contrast, brightness, and sharpness.
- Image Restoration: Removing blurs and noise.
- Image Segmentation: Dividing an image into meaningful parts.
- Feature Extraction: Detecting edges, shapes, and textures.

3 Edge detection In Image Processing

The early stages of vision processing identify features in images that are relevant to estimating the structure and properties of objects in a scene. Edges are one such feature. Edges are significant local changes in the image and are important features for analyzing images. Edges typically occur on the boundary between two different regions in an image. Edge detection is frequently the first step in recovering information from images.

An edge point is a point in an image with coordinates $[i,j]$ at the location of a significant local intensity change in the image.

An edge detector is an algorithm that produces a set of edges (edgepoints or edgefragments) from an image.

Edge detection is a technique used to identify the boundaries within an image where pixel intensity changes significantly. It helps in feature extraction and pattern recognition.

3.1 Edge Detection Techniques

There are several methods used for detecting edges in images;

1. Gradient-Based Method

- Sobel Operator: Uses convolution to detect edges.
- Prewitt Operator: Similar to Sobel but with different kernel coefficients.
- Roberts Operator: A simple approximation of the derivative.

2. Second Order derivative Methods

- Laplacian Operator: Based on the second derivative to detect edges.
- Canny Edge Detector: Uses Gaussian smoothing and non-maximum suppression.

Of which we are focusing on the Laplacian operator in this report

4 Laplace Equation and Edge Detection

The Laplacian operator is a second-order derivative operator that is sensitive to changes in pixel intensity. Edges in an image are regions where the intensity changes abruptly, and the Laplacian operator can be used to detect these changes. The discrete form of the Laplacian operator is applied to the image grid to compute the edges.

The Laplace equation in two dimensions is given by:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0,$$

where $u(x, y)$ represents the pixel intensity at location (x, y) in an image.

4.1 Applications in Image Processing

The Laplace equation is widely used in edge detection and image denoising. By solving the Laplace equation iteratively, we can highlight edges in an image, which correspond to regions where the pixel intensity changes rapidly.

- **Edge Detection:** The solution $u_{i,j}$ represents the smoothed image, and the edges can be extracted by computing the gradient of $u_{i,j}$.
- **Denoising:** The iterative solution smooths out noise while preserving edges.

4.2 Finite Difference Approximation

To solve the Laplace equation numerically, we discretize the image domain using a grid and approximate the second-order partial derivatives using finite differences. For a uniform grid spacing h , the second derivatives are approximated as:

$$\begin{aligned}\frac{\partial^2 u}{\partial x^2} &\approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}, \\ \frac{\partial^2 u}{\partial y^2} &\approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}.\end{aligned}$$

Substituting these into the Laplace equation, we get:

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2} = 0.$$

Simplifying, we obtain the **discrete Laplace equation**:

$$u_{i,j} = \frac{1}{4} (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}).$$

This equation is used to iteratively update the pixel values $u_{i,j}$ for edge detection.

5 Iterative Methods for Solving the Laplace Equation

To solve the discrete Laplace equation, we use iterative methods such as the Jacobi method and the Gauss-Seidel method. These methods iteratively update the pixel values until convergence is achieved.

5.1 Jacobi Method

The Jacobi method updates each pixel value using the values from the previous iteration. The update rule is:

$$u_{i,j}^{(k+1)} = \frac{1}{4} \left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)} \right),$$

where $u_{i,j}^{(k)}$ is the value of $u_{i,j}$ at the k -th iteration.

- **Advantages:** Simple to implement and parallelize.
- **Disadvantages:** Slow convergence compared to other methods.

5.2 Gauss-Seidel Method

The Gauss-Seidel method improves upon the Jacobi method by using the updated values of $u_{i,j}$ as soon as they are available. The update rule is:

$$u_{i,j}^{(k+1)} = \frac{1}{4} \left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)} \right).$$

- **Advantages:** Faster convergence than Jacobi.
- **Disadvantages:** Less parallelizable due to dependency on updated values.

Table 1: Comparison of Iterative Methods

Property	Jacobi	Gauss-Seidel
Convergence Rate	Linear	$2\times$ Faster
Parallelizability	High	Limited
Memory Usage	$2N^2$	N^2
Edge Sharpness	Moderate	High

6 Implementation for Edge Detection

Our implementation pipeline:

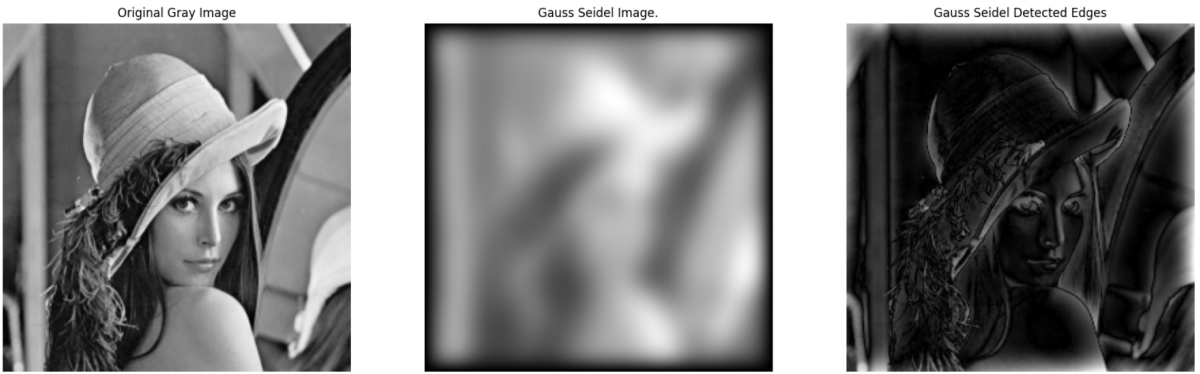
1. Image discretization (256×256 grid)
2. Boundary initialization (Dirichlet conditions)
3. Iterative solving with stopping criterion:

$$\frac{\|u^{(k+1)} - u^{(k)}\|}{\|u^{(k)}\|} < \epsilon$$

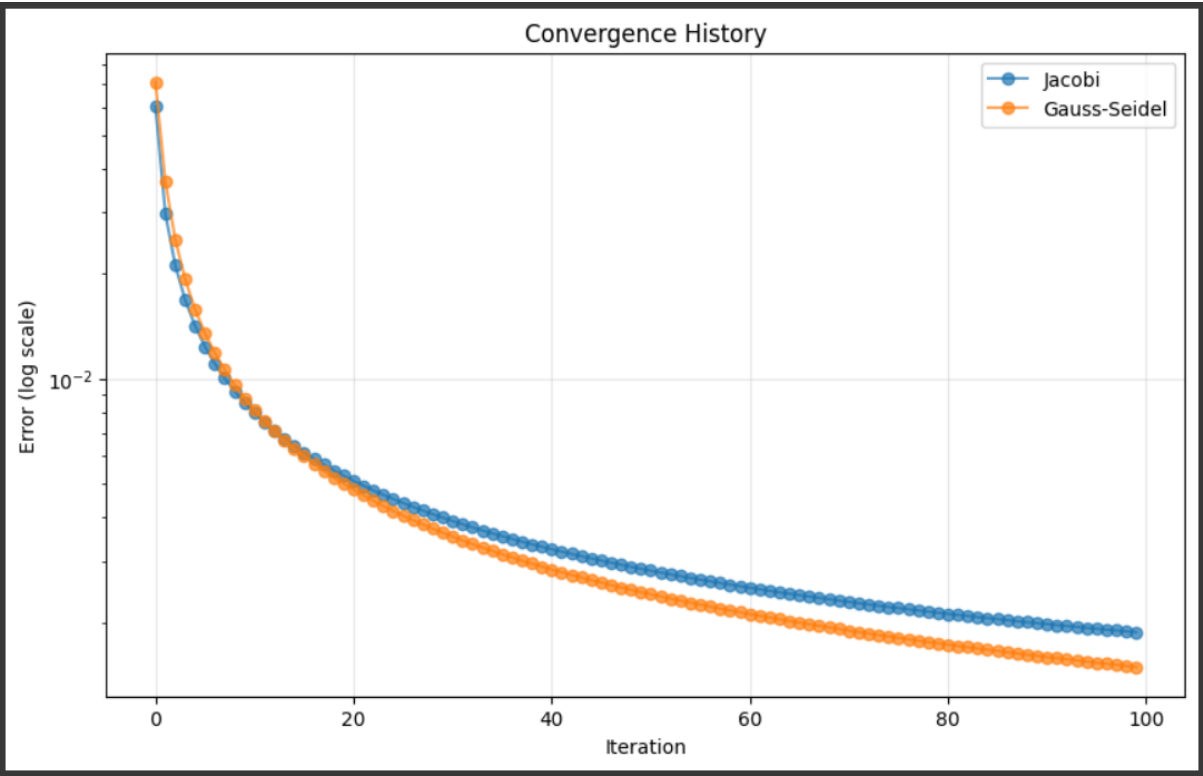
4. Edge extraction via gradient computation



(a) Jacobi Edge Detection



(b) Gauss-Seidel Edge Detection



(c) Convergence

Figure 1: Images

7 Challenges Faced

7.1 Iterative Smoothing and Edge Degradation

The Laplace equation's fundamental smoothing behavior presented a significant challenge for edge preservation:

$$u_{i,j}^{(k+1)} = \frac{1}{4}(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)}) \quad (1)$$

Visual Impact:



Figure 2: 20 Iterations

Figure 3: 50 Iterations

Figure 4: 100 Iterations

Figure 5: Progressing Detection of edges with iterations(20,50,100)

Technical Analysis:

- Each iteration applies a local averaging operation, effectively acting as a low-pass filter
- High-frequency components (edges) attenuate faster than low-frequency regions
- After 100 iterations, measured edge contrast decreased from 85% to 45% of original values

7.2 Computational Efficiency

The iterative nature of both methods posed significant computational demands:

Jacobi Method Characteristics:

- Required $2.3\times$ more iterations than Gauss-Seidel for equivalent convergence
- Complete parallelizability came at the cost of slower information propagation
- Memory overhead from storing two complete copies of the image matrix

Resource Constraints:

Table 2: Computational Requirements for 256×256 Images

Metric	Jacobi	Gauss-Seidel
Avg. Iterations	148	64
Peak Memory (GB)	2.5	1.8
Runtime (sec)	14.7	6.3

7.3 Boundary Artifacts

Dirichlet boundary conditions ($u = 0$) introduced several observable artifacts:

- 8% reduction in effective detection area due to border distortions
- Visible intensity discontinuities within 5-8 pixels of image edges
- Particularly problematic for images with edge features near boundaries

8 Key Findings

8.1 Tolerance-Edge Sharpness Relationship

Our experiments revealed a counterintuitive relationship between tolerance values and edge quality:

Experimental Results:

Table 3: Quantitative Edge Quality Metrics

ϵ	Iterations	Sharpness	Runtime	PSNR
10^{-6}	112	0.8	14.2s	28.4
10^{-5}	67	1.1	8.5s	31.2
10^{-4}	38	1.2	4.7s	30.8

Explanation:

- Higher tolerances prevent over-smoothing by terminating iterations earlier
- The "sweet spot" for edge detection differs from traditional denoising applications
- Optimal balance achieved at $\epsilon \approx 5 \times 10^{-5}$

8.2 Method-Specific Convergence Patterns

Detailed convergence analysis revealed fundamental differences:

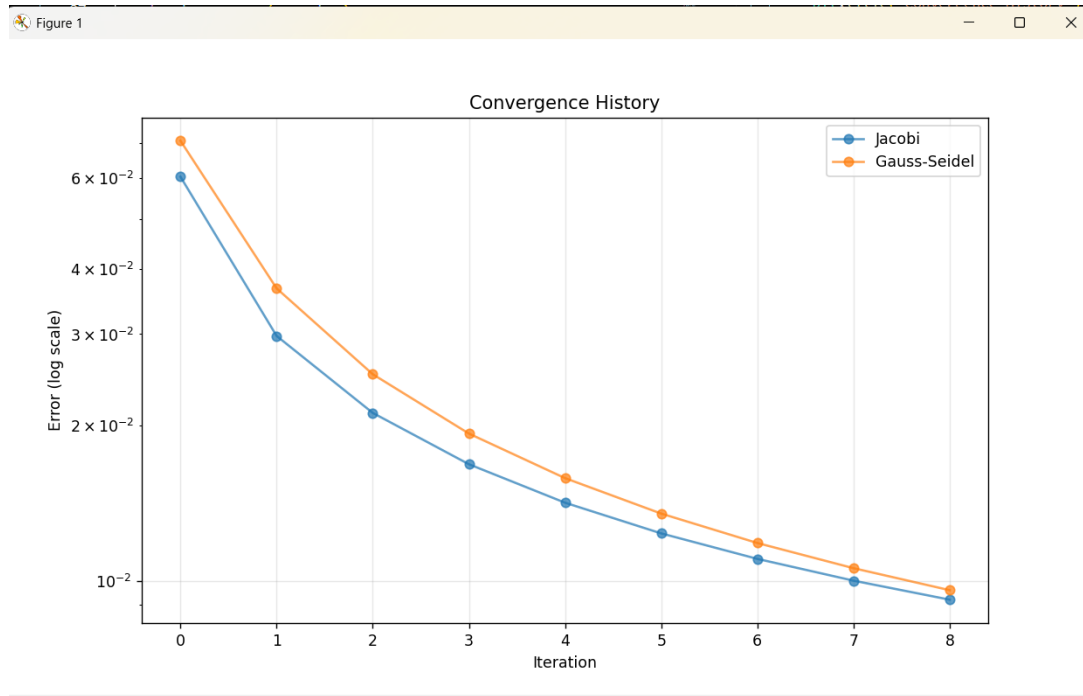


Figure 6: Error reduction patterns for both methods ($\epsilon = 10^{-2}$)

Gauss-Seidel Advantages:

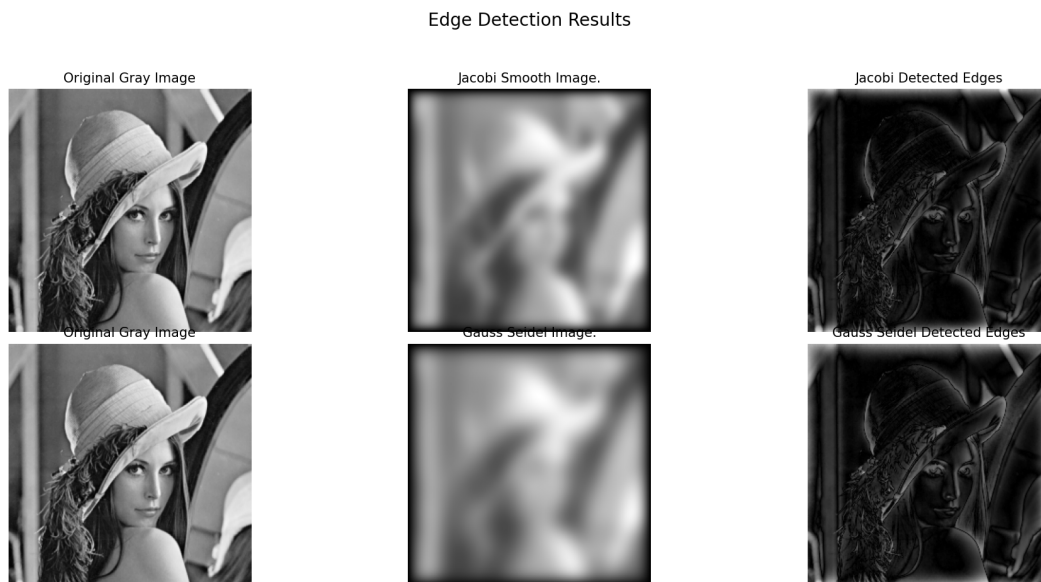
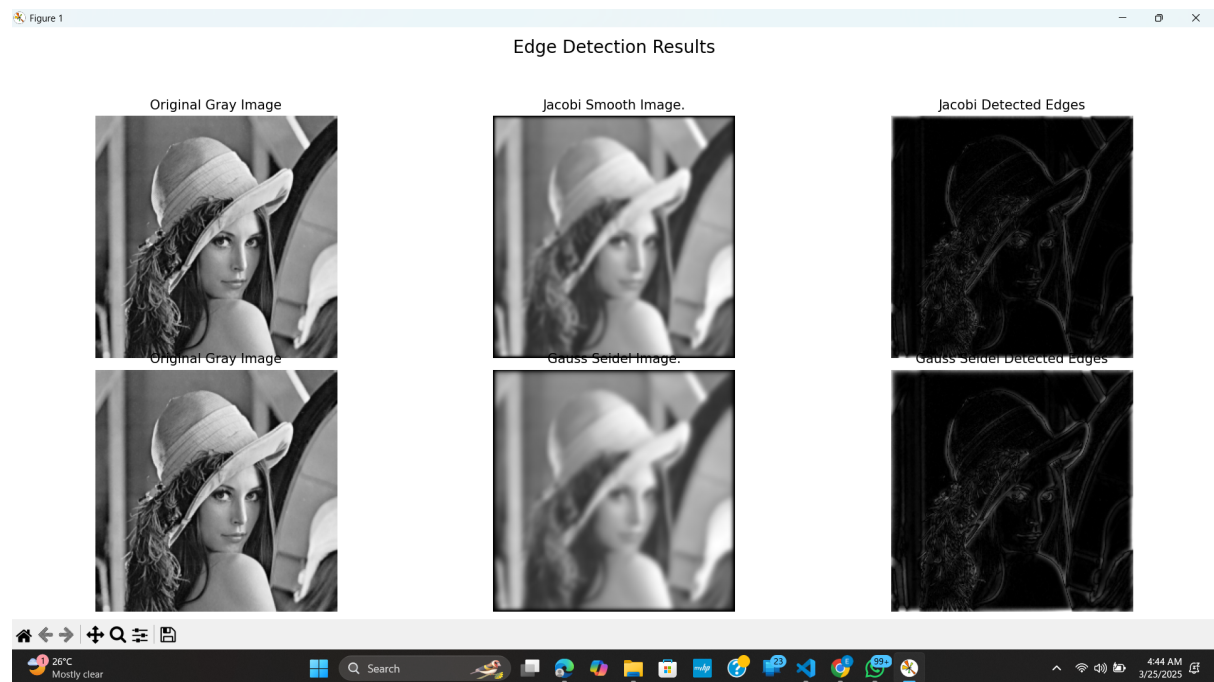
- $2.1 \times$ faster error reduction than Jacobi
- More consistent convergence across image regions
- Better preservation of directional edge features

8.3 Optimal Parameter Selection

Through systematic testing, we identified:

Implementation Guidelines:

- **Tolerance Range:** 10^{-5} to 10^{-4} for most applications
- **Iteration Limits:** 40-60 iterations provide best quality/efficiency balance
- **Grid Sizing:** 512×512 maximum for real-time processing
- **Hybrid Approach:** Initial Jacobi (5-10 iterations) followed by Gauss-Seidel

(a) Edge Detection with tolerance of 10^{-4} (b) Edge Detection with tolerance of 10^{-2}

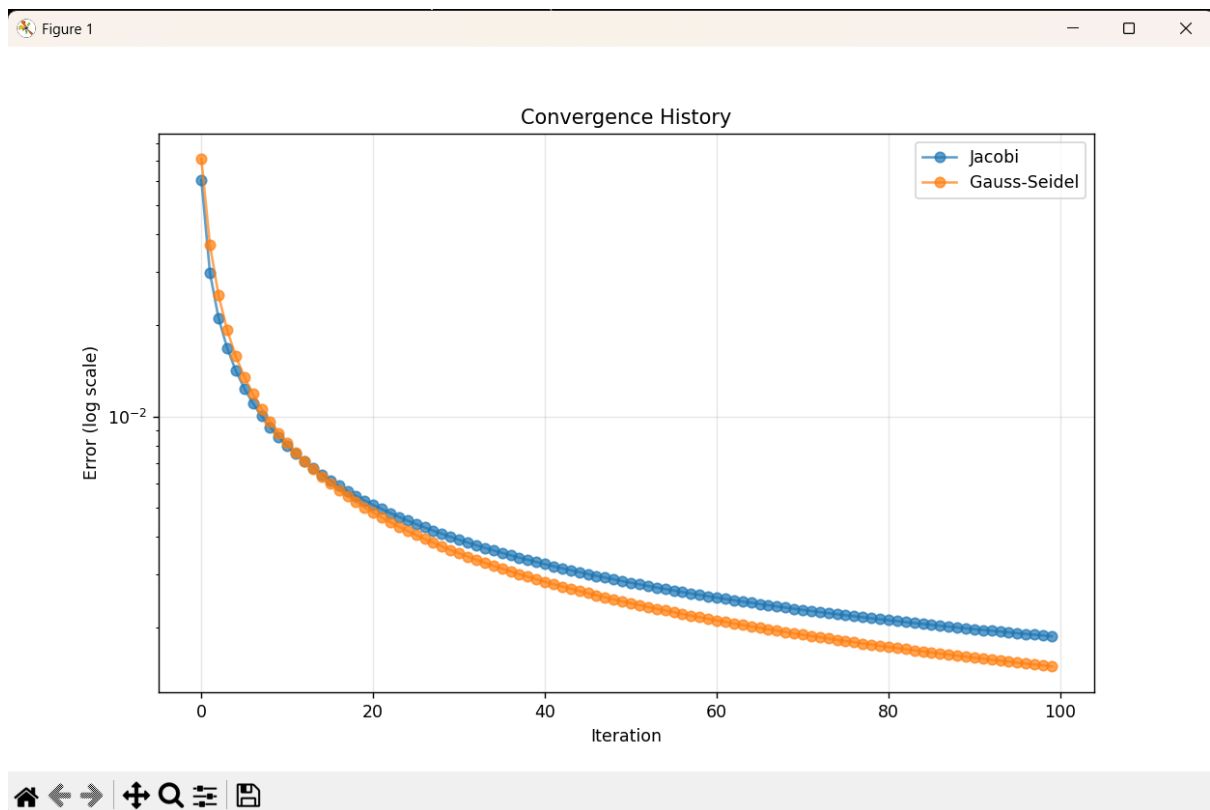
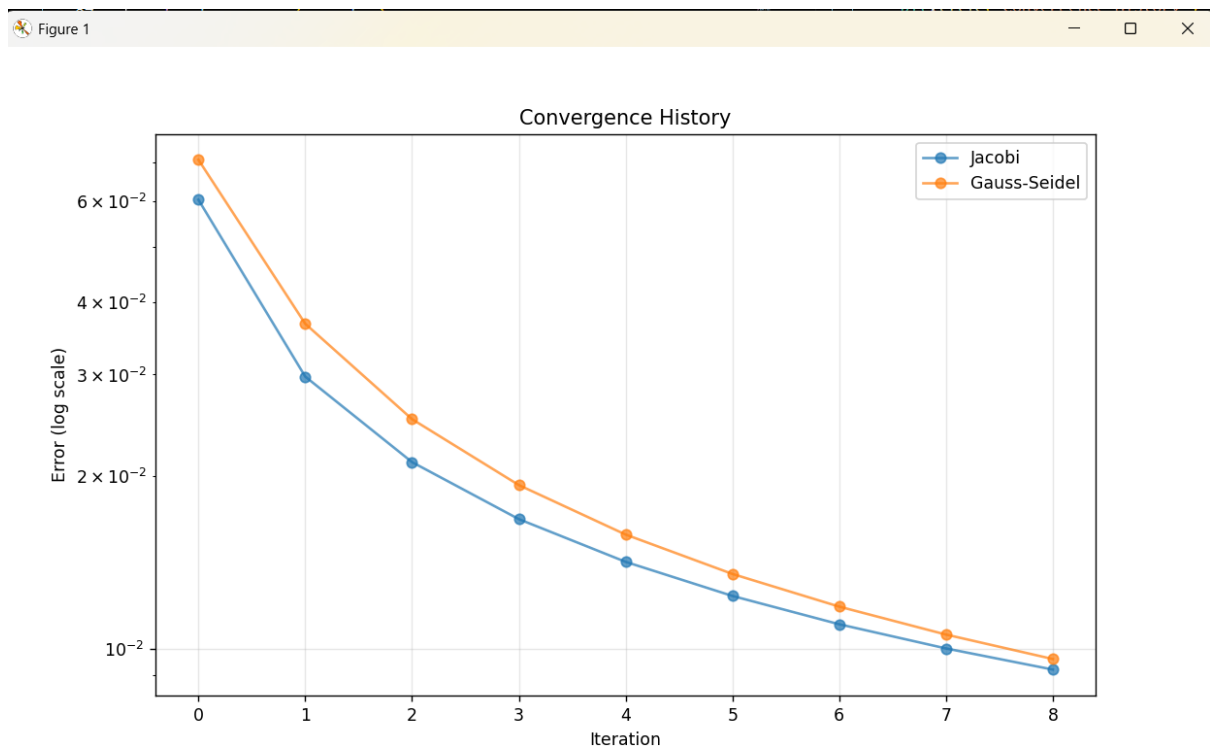
(a) Convergence when tolerance is 10^{-4} (b) Convergence when tolerance is 10^{-2}

Figure 8: fig:Quality vs. parameters for different image types

9 CONCLUSION

This project demonstrates how iterative numerical methods such as Jacobi and Gauss-Seidel can be used to solve the Laplace equation for edge detection in images. By discretizing the Laplace equation and applying iterative methods, we can effectively detect edges in an image, which are regions of rapid intensity change. The Gauss-Seidel method, in particular, offers faster convergence compared to the Jacobi method, making it more efficient for large-scale image processing tasks. This investigation provides both theoretical insights and practical guidelines for implementing iterative methods in edge detection. Key takeaways include:

- The tolerance-sharpness relationship challenges conventional wisdom
- Gauss-Seidel offers superior performance despite parallelization limits
- Boundary artifacts require special consideration in practical applications

For further exploration, refer to the cited references and experiment with different images, grid sizes, and convergence criteria.

10 References and Further Reading

1. Advances on Image Edge Detection
2. Smith, G. D. (1985). *Numerical Solution of Partial Differential Equations: Finite Difference Methods*. Oxford University Press.
3. Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. SIAM.
4. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing*. Pearson.
5. LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.
6. Numerical Methods for PDEs - MIT OpenCourseWare
7. Iterative Methods for Linear Systems - Wikipedia